Uninformed Search

Artificial Intelligence · Computer Science · Sofia University

Bogomil Stoyanov

© 2026 Bogomil Stoyanov

Outline

1. Formulating a search problem
2. Judging a strategy
3. One algorithm, many strategies
4. Breadth-first search
5. Uniform-cost search
6. Depth-first search
7. Depth-limited and iterative deepening search
8. Searching from both ends
9. Summary

© 2026 Bogomil Stoyanov

Formulating a search problem

© 2026 Bogomil Stoyanov

Defining a search problem

Search problem

1. an **initial state**, where we start;
2. a set of **actions** available in each state;
3. a **transition model**: which state an action leads to;
4. a **goal test**: is this state a solution?
5. a **step cost** $c(s, a, s') \geq 0$.

© 2026 Bogomil Stoyanov

Defining a search problem

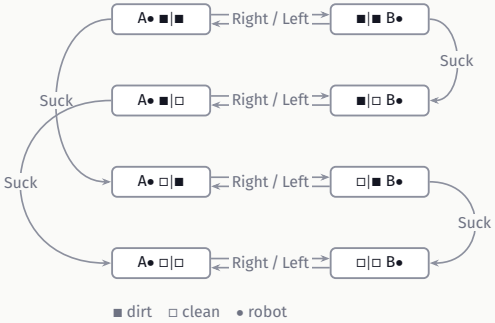
Search problem

1. an **initial state**, where we start;
2. a set of **actions** available in each state;
3. a **transition model**: which state an action leads to;
4. a **goal test**: is this state a solution?
5. a **step cost** $c(s, a, s') \geq 0$.

A **solution** is a *sequence of actions* leading from the initial state to a goal state. An **optimal** solution is one of least total cost. What is *not* here: no map of the space, no hint as to which direction is promising. That is what makes this search *uninformed*.

© 2026 Bogomil Stoyanov

Worked formulation: the vacuum world



States. Robot in *A* or *B*; each square dirty or clean. $2 \times 2 \times 2 = 8$ states, so the whole space fits on this slide.

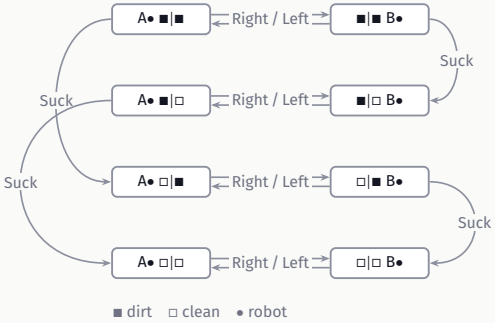
Actions. `Left`, `Right`, `Suck`.

Goal test. Both squares clean (two of the eight states).

Step cost. 1 per action.

© 2026 Bogomil Stoyanov

Worked formulation: the vacuum world



States. Robot in *A* or *B*; each square dirty or clean. $2 \times 2 \times 2 = \mathbf{8}$ states, so the whole space fits on this slide.

Actions. `Left`, `Right`, `Suck`.

Goal test. Both squares clean (two of the eight states).

Step cost. 1 per action.

Why start here? Because the state space is a *graph we can see*. For the puzzles on the next two slides it is far too large to draw, but the algorithms cannot tell the difference.

© 2026 Bogomil Stoyanov

Worked formulation: the 8-puzzle



initial state

goal state

States. A permutation of the 8 tiles plus the blank.

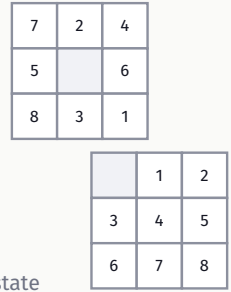
Actions. Move the *blank* up / down / left / right. Modelling the blank rather than the tiles turns 8 movable objects into one: the branching factor drops from around 24 to at most 4.

Goal test. State equals the goal arrangement.

Step cost. 1 per move.

© 2026 Bogomil Stoyanov

Worked formulation: the 8-puzzle



States. A permutation of the 8 tiles plus the blank.

Actions. Move the *blank* up / down / left / right. Modelling the blank rather than the tiles turns 8 movable objects into one: the branching factor drops from around 24 to at most 4.

Goal test. State equals the goal arrangement.

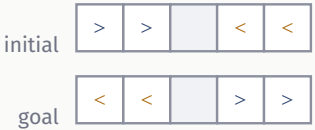
Step cost. 1 per move.

> $9!/2 = \mathbf{181\,440}$ reachable states, which is fine. The 15-puzzle has 1.3×10^{12}, and finding an *optimal* solution to the general n-puzzle is NP-hard. The formulation is easy; the search is not.

© 2026 Bogomil Stoyanov

Worked formulation: frog leap

$2N + 1$ squares, N frogs facing right, N facing left, one gap. Each frog moves only forwards, either stepping into the gap or jumping over one frog into it. Swap the two groups.



© 2026 Bogomil Stoyanov

Worked formulation: frog leap

$2N + 1$ squares, N frogs facing right, N facing left, one gap. Each frog moves only forwards, either stepping into the gap or jumping over one frog into it. Swap the two groups.

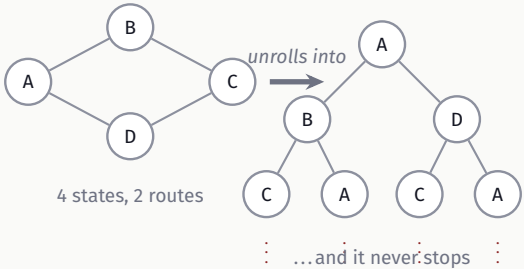


State. The string itself, e.g. »_«.
Actions. Step or jump, left-facing or right-facing: at most 4 legal in any state.
Goal test. String equals «_».

Every action is *reversible*, so the state space is an undirected graph with cycles. A search that does not remember where it has been will loop forever. Hold that thought.

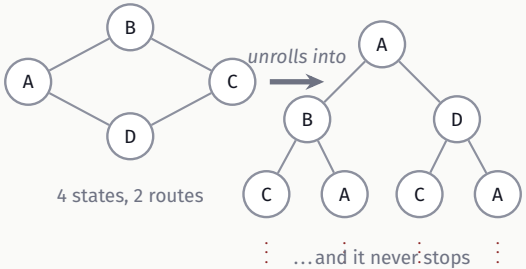
© 2026 Bogomil Stoyanov

The state space is not the search tree



© 2026 Bogomil Stoyanov

The state space is not the search tree



The **state space** is finite: 4 states.
The **search tree** it unrolls into is **infinite**: every cycle becomes an endless branch.

So a **node** is not a **state**. A state is a configuration of the world; a node is a *place in the search tree* that happens to hold a state. Two different nodes can hold the same state.

© 2026 Bogomil Stoyanov

Inside a node

NODE

```
1 state                ▷ a configuration
2 parent               ▷ the node we came from
3 action               ▷ what got us here
4 path-cost g          ▷ total cost from the start
```

`parent` is what makes the answer recoverable: the algorithms below return a *node*, and we read the solution off by walking `parent` pointers back to the start and reversing the result.

© 2026 Bogomil Stoyanov

Inside a node

NODE

```
1 state                ▷ a configuration
2 parent               ▷ the node we came from
3 action               ▷ what got us here
4 path-cost g          ▷ total cost from the start
```

`parent` is what makes the answer recoverable: the algorithms below return a *node*, and we read the solution off by walking `parent` pointers back to the start and reversing the result.

Every algorithm today uses two containers:

- the **frontier**: nodes generated but not yet expanded. *Which node we take from it is the only thing that differs between today's algorithms.*
- the **explored set**: states already expanded.

© 2026 Bogomil Stoyanov

Judging a strategy

© 2026 Bogomil Stoyanov

Evaluating a strategy

Complete?	If a solution exists, is it guaranteed to find one?
Optimal?	Is the solution it finds guaranteed to be of least cost?
Time	How many nodes does it generate?
Space	How many nodes does it hold in memory at once?

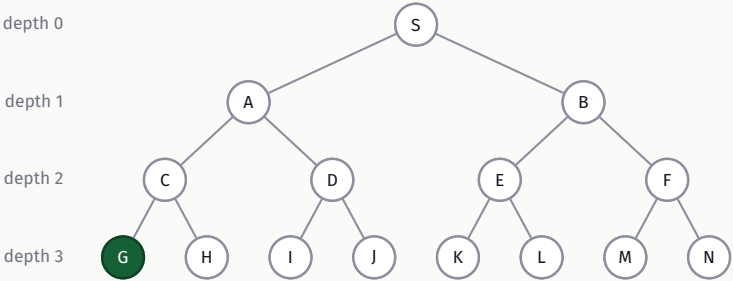
Time and space are measured in *nodes*, in terms of:

- b, the **branching factor**, the number of successors per node;
- d, the **depth** of the shallowest goal;
- m, the **maximum depth** of the space, possibly ∞.

> **Complete** and **optimal** are different promises. An algorithm can be guaranteed to find *an* answer and still reliably return a bad one. We will see exactly that twice today.

© 2026 Bogomil Stoyanov

b, d and m on a concrete tree



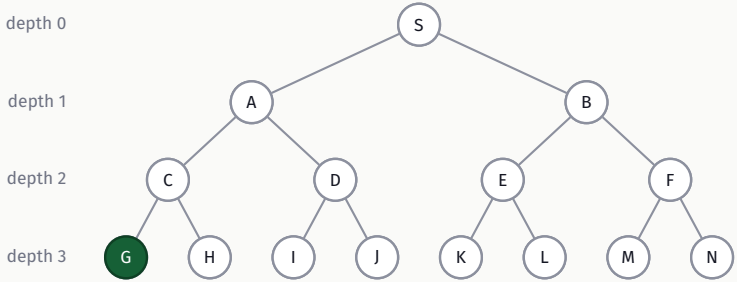
$b = 2$: every node has 2 successors.

$d = 3$: the goal sits at depth 3.

$m = 3$: nothing is deeper.

© 2026 Bogomil Stoyanov

b, d and m on a concrete tree



$b = 2$: every node has 2 successors.

$d = 3$: the goal sits at depth 3.

$m = 3$: nothing is deeper.

A tree with $b = 2$, $d = 3$ has $1 + 2 + 4 + 8 = 15$ nodes. With $b = 10$, $d = 12$ it has 1.1×10^{12}. Every algorithm today is a way of not visiting all of them.

© 2026 Bogomil Stoyanov

One algorithm, many strategies

© 2026 Bogomil Stoyanov

The generic search loop

GRAPH-SEARCH(PROBLEM)

```
1  frontier ← { NODE(problem.initial) }
2  explored ← { }
3  loop
4      if frontier is empty then return failure
5      node ← REMOVE(frontier)                ▷ the strategy
6      if GOAL?(node.state) then return node
7      add node.state to explored
8      for each child of node
9          if child.state not in explored ∪ frontier
10             add child to frontier
```

© 2026 Bogomil Stoyanov

The generic search loop

GRAPH-SEARCH(PROBLEM)

```
1  frontier ← { NODE(problem.initial) }
2  explored ← {}
3  loop
4      if frontier is empty then return failure
5      node ← REMOVE(frontier)                ▷ the strategy
6      if GOAL?(node.state) then return node
7      add node.state to explored
8      for each child of node
9          if child.state not in explored ∪ frontier
10             add child to frontier
```

Line 5 is where the three algorithms differ.

- the *oldest* node ⇒ **breadth-first**
- the *cheapest* node ⇒ **uniform-cost**
- the *newest* node ⇒ **depth-first**

© 2026 Bogomil Stoyanov

The generic search loop

GRAPH-SEARCH(PROBLEM)

```
1  frontier ← { NODE(problem.initial) }
2  explored ← { }
3  loop
4      if frontier is empty then return failure
5      node ← REMOVE(frontier)                    ▷ the strategy
6      if GOAL?(node.state) then return node
7      add node.state to explored
8      for each child of node
9          if child.state not in explored ∪ frontier
10             add child to frontier
```

Line 5 is where the three algorithms differ.

- the *oldest* node ⇒ **breadth-first**
- the *cheapest* node ⇒ **uniform-cost**
- the *newest* node ⇒ **depth-first**

So the next three algorithms are all this one, with line 5, and occasionally line 6, rewritten. Each slide shows only what changes.

© 2026 Bogomil Stoyanov

Tree search: the loop without an explored set

TREE-SEARCH(PROBLEM)

```
1 frontier ← { NODE(problem.initial) }
2 loop
3     if frontier is empty then return failure
4     node ← REMOVE(frontier)
5     if GOAL?(node.state) then return node
6     for each child of node
7         add child to frontier
```

Simpler, and shorter by two lines. It is also *wrong* on every problem we formulated earlier.

© 2026 Bogomil Stoyanov

Tree search: the loop without an explored set

TREE-SEARCH(PROBLEM)

```
1 frontier ← { NODE(problem.initial) }
2 loop
3     if frontier is empty then return failure
4     node ← REMOVE(frontier)
5     if GOAL?(node.state) then return node
6     for each child of node
7         add child to frontier
```

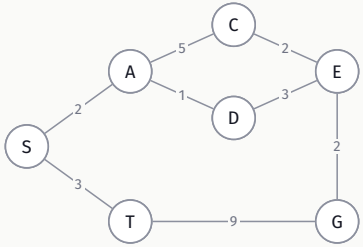
Simpler, and shorter by two lines. It is also *wrong* on every problem we formulated earlier.

On the frog-leap graph every action is reversible, so tree search walks »_« → ><_>< → »_« → ⋯ forever.

> The explored set costs memory and buys **termination**. On a graph with cycles it is not optional, and every interesting state space has cycles.

© 2026 Bogomil Stoyanov

The graph we will use throughout the session

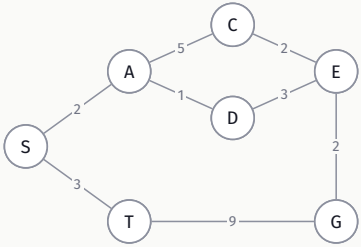


Start *S*, goal *G*. Successors are always expanded in **alphabetical order**, so every trace today is reproducible by hand, and so is your implementation's.

Three paths lead from *S* to *G*. Before we start: which one do you expect a search to return?

© 2026 Bogomil Stoyanov

The graph we will use throughout the session



Start *S*, goal *G*. Successors are always expanded in **alphabetical order**, so every trace today is reproducible by hand, and so is your implementation's.

Three paths lead from *S* to *G*. Before we start: which one do you expect a search to return?

It depends entirely on which search you ask. By the end of the session this graph will have given us three different answers.

© 2026 Bogomil Stoyanov

Breadth-first search

© 2026 Bogomil Stoyanov

Breadth-first search: take the oldest node

BFS = GRAPH-SEARCH, WITH:

```
 5  node ← POP( FIFO queue )
 6  deleted; instead test each child as it is
    generated:
10  if GOAL?(child.state) then return child
    add child to frontier
```

A FIFO queue expands nodes in the order they were generated, so all of depth k is expanded before anything at depth $k+1$.

© 2026 Bogomil Stoyanov

Breadth-first search: take the oldest node

BFS = GRAPH-SEARCH, WITH:

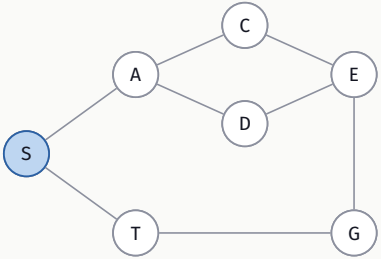
```
5  node ← POP( FIFO queue )
6  deleted; instead test each child as it is
   generated:
10 if GOAL?(child.state) then return child
   add child to frontier
```

A FIFO queue expands nodes in the order they were generated, so all of depth k is expanded before anything at depth $k+1$.

> Testing on *generation* rather than on *removal* saves a whole level of expansion: BFS would otherwise generate $O(b^{d+1})$ nodes. It is safe only because every step costs the same. Uniform-cost search will have to move the test back.

© 2026 Bogomil Stoyanov

BFS, step by step



FRONTIER: FIFO QUEUE

S

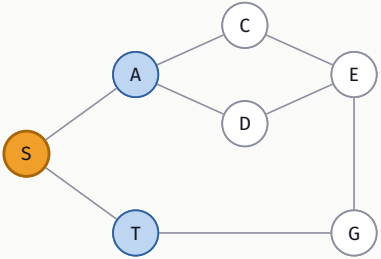
EXPLORED

empty

Start: *S* alone in the frontier.

© 2026 Bogomil Stoyanov

BFS, step by step



FRONTIER: FIFO QUEUE

A	T

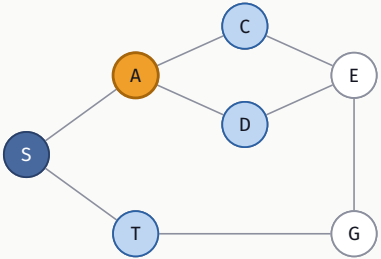
EXPLORED

{S}

Expand *S*. Generate *A* and *T*; neither is the goal.

© 2026 Bogomil Stoyanov

BFS, step by step



FRONTIER: FIFO QUEUE

T	C	D

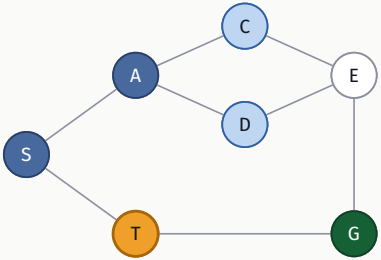
EXPLORED

{S, A}

Expand the *oldest* node, *A*. Generate *C* and *D*.

© 2026 Bogomil Stoyanov

BFS, step by step



FRONTIER: FIFO QUEUE

C | D

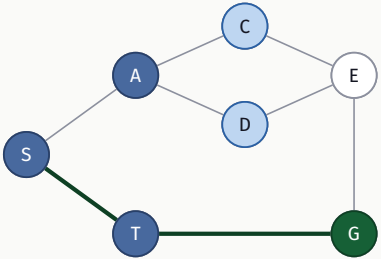
EXPLORED

{S, A, T}

Expand *T*, and its child *G is* the goal. Stop.

© 2026 Bogomil Stoyanov

BFS, step by step



FRONTIER: FIFO QUEUE

C | D

EXPLORED

{S, A, T}

BFS returns $S \to T \to G$**: 2 edges, total cost** $3 + 9 = 12$**.**

© 2026 Bogomil Stoyanov

Breadth-first search: the verdict

© 2026 Bogomil Stoyanov

Breadth-first search: the verdict

Complete? Yes, provided b is finite. It cannot miss a goal at depth d, because it expands every node above that depth first.

© 2026 Bogomil Stoyanov

Breadth-first search: the verdict

Complete? Yes, provided b is finite. It cannot miss a goal at depth d, because it expands every node above that depth first.

Optimal? **Only if every step costs the same.** BFS minimises the *number of edges*, which is not the same as cost. On our graph it returned a 2-edge path costing 12 when a path of cost 8 exists.

© 2026 Bogomil Stoyanov

Breadth-first search: the verdict

Complete?	Yes, provided b is finite. It cannot miss a goal at depth d, because it expands every node above that depth first.
Optimal?	**Only if every step costs the same.** BFS minimises the *number of edges*, which is not the same as cost. On our graph it returned a 2-edge path costing 12 when a path of cost 8 exists.
Time	$O(b^d)$ generated nodes.

© 2026 Bogomil Stoyanov

Breadth-first search: the verdict

Complete?	Yes, provided b is finite. It cannot miss a goal at depth d, because it expands every node above that depth first.
Optimal?	**Only if every step costs the same.** BFS minimises the *number of edges*, which is not the same as cost. On our graph it returned a 2-edge path costing 12 when a path of cost 8 exists.
Time	$O(b^d)$ generated nodes.
Space	$O(b^d)$, and *this* is what kills it. The entire frontier is in memory at once.

© 2026 Bogomil Stoyanov

The memory cost

Generating 10^6 nodes/second, 1 kB per node, $b = 10$:

Depth d	**Nodes**	**Memory**
2	110	110 kB
4	11 110	11 MB
6	10^6	1 GB
8	10^8	100 GB
10	10^{10}	10 TB
12	10^{12}	**1 PB**

© 2026 Bogomil Stoyanov

The memory cost

Generating 10^6 nodes/second, 1 kB per node, $b = 10$:

Depth d	Nodes	Memory
2	110	110 kB
4	11 110	11 MB
6	10^6	1 GB
8	10^8	100 GB
10	10^{10}	10 TB
12	10^{12}	**1 PB**

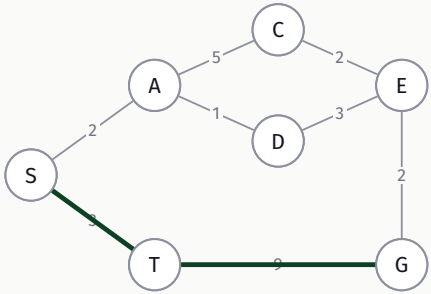
Depth 12 would take about 13 days of CPU, annoying but survivable, and a *petabyte* of RAM, which is not. For BFS, memory runs out long before patience does. Remember this when we get to iterative deepening.

© 2026 Bogomil Stoyanov

Uniform-cost search

© 2026 Bogomil Stoyanov

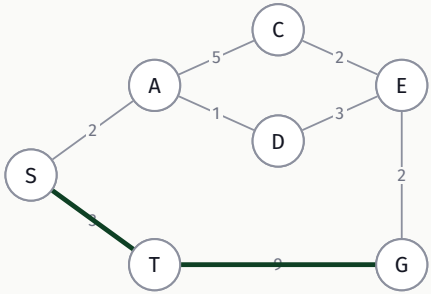
BFS gave us cost 12. Can we do better?



BFS chose $S \to T \to G$ because it has the *fewest edges*. But that route uses the single most expensive edge in the graph ($T \to G$, cost 9).

© 2026 Bogomil Stoyanov

BFS gave us cost 12. Can we do better?

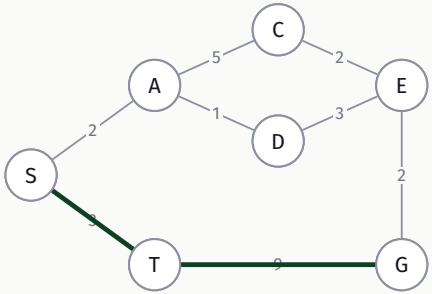


BFS chose $S \to T \to G$ because it has the *fewest edges*. But that route uses the single most expensive edge in the graph ($T \to G$, cost 9).$S \to A \to C \to E \to G$ costs $2 + 5 + 2 + 2 = 11$.

$S \to A \to D \to E \to G$ costs $2 + 1 + 3 + 2 = \mathbf{8}$.

© 2026 Bogomil Stoyanov

BFS gave us cost 12. Can we do better?



BFS chose $S \to T \to G$ because it has the *fewest edges*. But that route uses the single most expensive edge in the graph ($T \to G$, cost 9).$S \to A \to C \to E \to G$ costs $2 + 5 + 2 + 2 = 11$.

$S \to A \to D \to E \to G$ costs $2 + 1 + 3 + 2 = \mathbf{8}$.

> Fix: order the frontier by **path cost** $g(n)$ instead of by depth. That one change is **uniform-cost search**.

© 2026 Bogomil Stoyanov

Uniform-cost search: take the cheapest node

UCS = GRAPH-SEARCH, WITH:

```
5  node ← POP-MIN( priority queue on g )
6  unchanged: the goal test stays here
9  else if child.state in frontier with higher g
       replace that frontier node by child
```

The goal test must *not* move. A goal that has just been generated may still be reachable more cheaply, so we may only accept it when it is popped as the minimum.

© 2026 Bogomil Stoyanov

Uniform-cost search: take the cheapest node

UCS = GRAPH-SEARCH, WITH:

5 node ← POP-MIN(priority queue on g)

6 unchanged: the goal test stays *here*

9 **else if** child.state **in** frontier **with** higher g

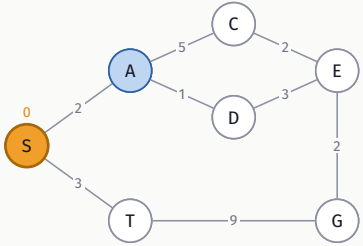
replace that frontier node by child

The goal test must *not* move. A goal that has just been generated may still be reachable more cheaply, so we may only accept it when it is popped as the minimum.

And the new `else if` case matters. If a cheaper route to a frontier node appears, its g must come down. Skip that and UCS stops being optimal. You will see the step it depends on in a moment.

© 2026 Bogomil Stoyanov

UCS, step by step: watch g



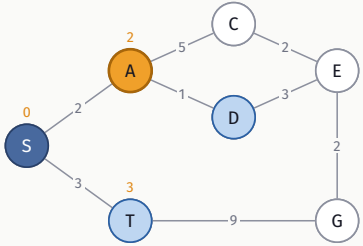
FRONTIER: PRIORITY QUEUE ON g

`S(0)`

Frontier: S at $g = 0$.

© 2026 Bogomil Stoyanov

UCS, step by step: watch g



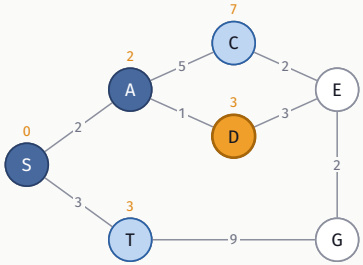
FRONTIER: PRIORITY QUEUE ON g

A(2)	T(3)

Expand *S*. *A* at 2, *T* at 3.

© 2026 Bogomil Stoyanov

UCS, step by step: watch g



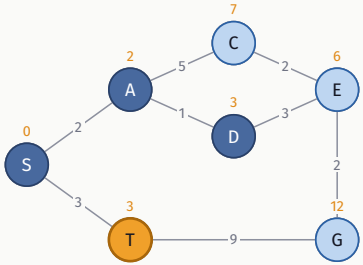
FRONTIER: PRIORITY QUEUE ON g

D(3)	T(3)	C(7)

Expand the cheapest, $A(2)$. C at 7, D at 3.

© 2026 Bogomil Stoyanov

UCS, step by step: watch g



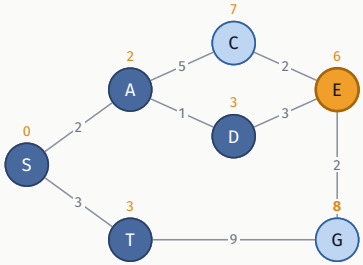
FRONTIER: PRIORITY QUEUE ON g

T(3)	E(6)	C(7)

A tie: D and T both cost 3. Break ties deterministically or your output is not reproducible. Take D: it gives E at 6.

© 2026 Bogomil Stoyanov

UCS, step by step: watch g



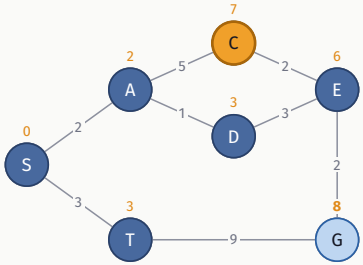
FRONTIER: PRIORITY QUEUE ON g

E(6)	C(7)	G(12)

Expand $T(3)$: it reaches the goal, G at 12. **We do not stop**: G was generated, not popped.

© 2026 Bogomil Stoyanov

UCS, step by step: watch g



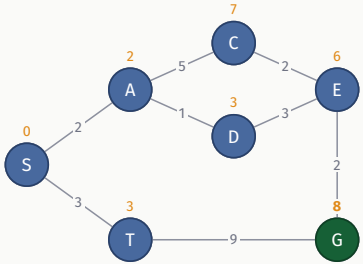
FRONTIER: PRIORITY QUEUE ON g

C(7)	G(8)

Expand $E(6)$. It reaches G at $6 + 2 = 8 < 12$, so **relax G from 12 down to 8**. This is the step people forget.

© 2026 Bogomil Stoyanov

UCS, step by step: watch g



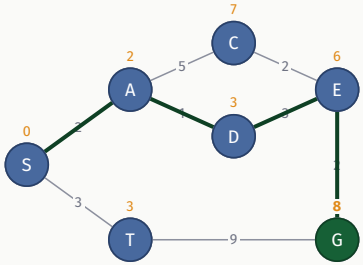
FRONTIER: PRIORITY QUEUE ON g

`G(8)`

Expand $C(7)$. It offers E at 9, worse than 6. Discard.

© 2026 Bogomil Stoyanov

UCS, step by step: watch g



FRONTIER: PRIORITY QUEUE ON g

`G(8)`

Now $G(8)$ is the cheapest node in the frontier. Pop it. **UCS returns $S \to A \to D \to E \to G$, cost 8.**

© 2026 Bogomil Stoyanov

Uniform-cost search: the verdict

© 2026 Bogomil Stoyanov

Uniform-cost search: the verdict

Complete? Yes, provided every step cost is at least some $\varepsilon > 0$. With zero-cost edges it can loop forever without ever increasing g.

© 2026 Bogomil Stoyanov

Uniform-cost search: the verdict

Complete? Yes, provided every step cost is at least some $\varepsilon > 0$. With zero-cost edges it can loop forever without ever increasing g.

Optimal? **Yes**, always, for any non-negative costs. It pops nodes in non-decreasing order of g, so the first goal popped is the cheapest.

© 2026 Bogomil Stoyanov

Uniform-cost search: the verdict

Complete?	Yes, provided every step cost is at least some $\varepsilon > 0$. With zero-cost edges it can loop forever without ever increasing g.
Optimal?	**Yes**, always, for any non-negative costs. It pops nodes in non-decreasing order of g, so the first goal popped is the cheapest.
Time	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$, where C^* is the optimal cost. Can be much worse than $O(b^d)$: many cheap steps are explored before one expensive one.

© 2026 Bogomil Stoyanov

Uniform-cost search: the verdict

Complete?	Yes, provided every step cost is at least some $\varepsilon > 0$. With zero-cost edges it can loop forever without ever increasing g.
Optimal?	**Yes**, always, for any non-negative costs. It pops nodes in non-decreasing order of g, so the first goal popped is the cheapest.
Time	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$, where C^* is the optimal cost. Can be much worse than $O(b^d)$: many cheap steps are explored before one expensive one.
Space	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$: same problem as BFS, same cause.

© 2026 Bogomil Stoyanov

Uniform-cost search: the verdict

Complete?	Yes, provided every step cost is at least some $\varepsilon > 0$. With zero-cost edges it can loop forever without ever increasing g.
Optimal?	**Yes**, always, for any non-negative costs. It pops nodes in non-decreasing order of g, so the first goal popped is the cheapest.
Time	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$, where C^* is the optimal cost. Can be much worse than $O(b^d)$: many cheap steps are explored before one expensive one.
Space	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$: same problem as BFS, same cause.

With every step cost equal, $C^*/\varepsilon = d$ and UCS degenerates into BFS, which is exactly why BFS is optimal in that one case.

© 2026 Bogomil Stoyanov

Depth-first search

© 2026 Bogomil Stoyanov

Depth-first search: take the newest node

DFS = GRAPH-SEARCH, WITH:

```
5  node ← POP( LIFO stack )
```

One word: *queue* became *stack*. Nothing else.

© 2026 Bogomil Stoyanov

Depth-first search: take the newest node

DFS = GRAPH-SEARCH, WITH:

```
5  node ← POP( LIFO stack )
```

One word: *queue* became *stack*. Nothing else. In practice you write it recursively and let the call stack *be* the frontier:

© 2026 Bogomil Stoyanov

Depth-first search: take the newest node

DFS = GRAPH-SEARCH, WITH:

```
5  node ← POP( LIFO stack )
```

One word: *queue* became *stack*. Nothing else. In practice you write it recursively and let the call stack *be* the frontier:

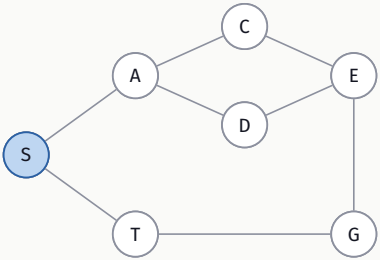
DFS(NODE)

```
1  if GOAL?(node.state) then return node
2  add node.state to explored
3  for each child of node
4      if child.state not in explored
5          r ← DFS(child)
6          if r ≠ failure then return r
7  return failure
```

Push children in *reverse* order in the iterative version if you want the two to visit nodes in the same order.

© 2026 Bogomil Stoyanov

DFS, step by step: same graph, same successor order



FRONTIER: LIFO STACK (TOP FIRST)

S

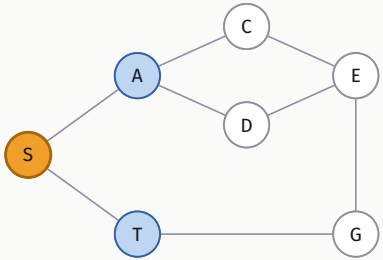
EXPLORED

empty

Start: *S* on the stack.

© 2026 Bogomil Stoyanov

DFS, step by step: same graph, same successor order



FRONTIER: LIFO STACK (TOP FIRST)

A	T

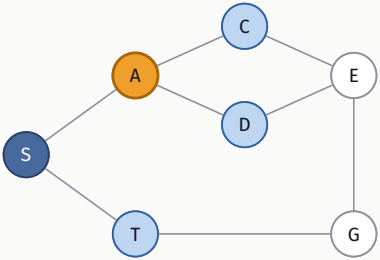
EXPLORED

{S}

Expand *S*, push *A* and *T*. *A* is on top.

© 2026 Bogomil Stoyanov

DFS, step by step: same graph, same successor order



FRONTIER: LIFO STACK (TOP FIRST)

C	D	T

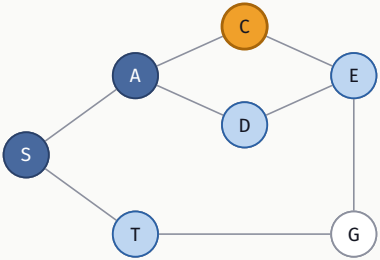
EXPLORED

{S, A}

Expand *A*. We go deep, not wide. Push *C*, *D*.

© 2026 Bogomil Stoyanov

DFS, step by step: same graph, same successor order



FRONTIER: LIFO STACK (TOP FIRST)

E	D	T

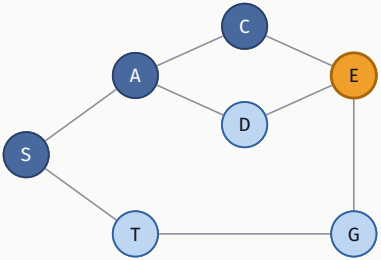
EXPLORED

{S, A, C}

Expand *C*. Push *E*.

© 2026 Bogomil Stoyanov

DFS, step by step: same graph, same successor order



FRONTIER: LIFO STACK (TOP FIRST)

D	G	D	T

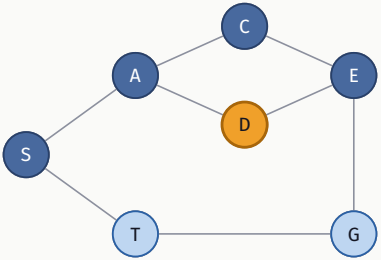
EXPLORED

{S, A, C, E}

Expand *E*. Push *D* and *G*. *D* **is now on the stack twice**: a state can sit in the frontier more than once; the explored set stops us *expanding* it twice.

© 2026 Bogomil Stoyanov

DFS, step by step: same graph, same successor order



FRONTIER: LIFO STACK (TOP FIRST)

G	D	T

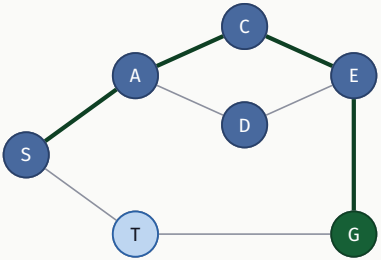
EXPLORED

{S, A, C, E, D}

Expand *D*: both its neighbours are already explored. Nothing new.

© 2026 Bogomil Stoyanov

DFS, step by step: same graph, same successor order



FRONTIER: LIFO STACK (TOP FIRST)

D	T

EXPLORED

{S, A, C, E, D}

Pop *G*. Goal. **DFS returns** $S \to A \to C \to E \to G$**: 4 edges, cost 11.** Not the shortest, and not the cheapest, but the first one found.

© 2026 Bogomil Stoyanov

Depth-first search: the verdict

© 2026 Bogomil Stoyanov

Depth-first search: the verdict

Complete? **Only on finite spaces, and only with the explored set.** On an infinite space it can descend one branch forever and never come back.

© 2026 Bogomil Stoyanov

Depth-first search: the verdict

Complete? **Only on finite spaces, and only with the explored set.** On an infinite space it can descend one branch forever and never come back.

Optimal? No. It returns the first goal it stumbles into: cost 11 here, where the best is 8.

© 2026 Bogomil Stoyanov

Depth-first search: the verdict

Complete?	**Only on finite spaces, and only with the explored set.** On an infinite space it can descend one branch forever and never come back.
Optimal?	No. It returns the first goal it stumbles into: cost 11 here, where the best is 8.
Time	$O(b^m)$, governed by the *maximum* depth m, not by d. If $m \gg d$ this is far worse than BFS.

© 2026 Bogomil Stoyanov

Depth-first search: the verdict

Complete?	**Only on finite spaces, and only with the explored set.** On an infinite space it can descend one branch forever and never come back.
Optimal?	No. It returns the first goal it stumbles into: cost 11 here, where the best is 8.
Time	$O(b^m)$, governed by the *maximum* depth m, not by d. If $m \gg d$ this is far worse than BFS.
Space	$O(bm)$, and this is the whole reason we tolerate it. It holds one path plus its siblings, not an entire level.

© 2026 Bogomil Stoyanov

DFS on an infinite space

A tiny infinite problem

From the integer N, reach the integer T.
Actions: $x \mapsto x - 1$ and $x \mapsto x \cdot 2$.
Take $N = 3$, $T = 5$.

© 2026 Bogomil Stoyanov

DFS on an infinite space

A tiny infinite problem

From the integer N, reach the integer T.
Actions: $x \mapsto x - 1$ and $x \mapsto x \cdot 2$.
Take $N = 3$, $T = 5$.

BFS finds $3 \to 6 \to 5$ in two steps.

DFS, if it tries $x \cdot 2$ first, walks
$3 \to 6 \to 12 \to 24 \to 48 \to \cdots$
and never returns.

© 2026 Bogomil Stoyanov

DFS on an infinite space

A tiny infinite problem

From the integer N, reach the integer T.
Actions: $x \mapsto x - 1$ and $x \mapsto x \cdot 2$.
Take $N = 3$, $T = 5$.

BFS finds $3 \to 6 \to 5$ in two steps.

DFS, if it tries $x \cdot 2$ first, walks
$3 \to 6 \to 12 \to 24 \to 48 \to \cdots$
and never returns.

The explored set does not save us here. It only stops us revisiting a state we have *already* seen, and this branch never revisits anything. It just keeps generating fresh, ever-larger integers.

© 2026 Bogomil Stoyanov

DFS on an infinite space

A tiny infinite problem

From the integer N, reach the integer T.
Actions: $x \mapsto x - 1$ and $x \mapsto x \cdot 2$.
Take $N = 3$, $T = 5$.

BFS finds $3 \to 6 \to 5$ in two steps.

DFS, if it tries $x \cdot 2$ first, walks
$3 \to 6 \to 12 \to 24 \to 48 \to \cdots$
and never returns.

The explored set does not save us here. It only stops us revisiting a state we have *already* seen, and this branch never revisits anything. It just keeps generating fresh, ever-larger integers.

So DFS's completeness problem comes from *unbounded depth* rather than from cycles. The fix is to bound the depth, which is what the next section does.

© 2026 Bogomil Stoyanov

Depth-limited and iterative deepening search

© 2026 Bogomil Stoyanov

Depth-limited search: DFS with a ceiling

DLS(NODE, PROBLEM, ℓ)

```
1 if GOAL?(node.state) then return node
2 if ℓ = 0 then return cutoff
3 cut ← false
4 for each child of node
5       r ← DLS(child, problem, ℓ − 1)
6       if r = cutoff then cut ← true
7       else if r ≠ failure then return r
8 return cut ? cutoff : failure
```

© 2026 Bogomil Stoyanov

Depth-limited search: DFS with a ceiling

DLS(NODE, PROBLEM, ℓ)

```
1 if GOAL?(node.state) then return node
2 if ℓ = 0 then return cutoff
3 cut ← false
4 for each child of node
5       r ← DLS(child, problem, ℓ − 1)
6       if r = cutoff then cut ← true
7       else if r ≠ failure then return r
8 return cut ? cutoff : failure
```

cutoff and **failure** must be different answers. *failure* means “no solution exists”. *cutoff* means “none within ℓ, though there might be one deeper”. Collapse them into one and the caller cannot tell whether to try again.

© 2026 Bogomil Stoyanov

Depth-limited search: DFS with a ceiling

DLS(NODE, PROBLEM, ℓ)

```
1 if GOAL?(node.state) then return node
2 if ℓ = 0 then return cutoff
3 cut ← false
4 for each child of node
5     r ← DLS(child, problem, ℓ − 1)
6     if r = cutoff then cut ← true
7     else if r ≠ failure then return r
8 return cut ? cutoff : failure
```

cutoff and **failure** must be different answers. *failure* means "no solution exists". *cutoff* means "none within ℓ, though there might be one deeper". Collapse them into one and the caller cannot tell whether to try again.

Now DFS terminates. But we have traded one problem for two:

- $\ell < d \Rightarrow$ *incomplete*: we cut above the goal.
- $\ell > d \Rightarrow$ still *not optimal*.

And usually we do not know d in advance.

© 2026 Bogomil Stoyanov

Iterative deepening: try every depth limit

ITERATIVE-DEEPENING-SEARCH(PROBLEM)

```
1 for ℓ = 0, 1, 2, . . .
2     r ← DLS(root, problem, ℓ)
3     if r ≠ cutoff then return r
```

Three lines. Run depth-limited search at every limit in turn, and throw the whole search away each time.

© 2026 Bogomil Stoyanov

Iterative deepening: try every depth limit

ITERATIVE-DEEPENING-SEARCH(PROBLEM)

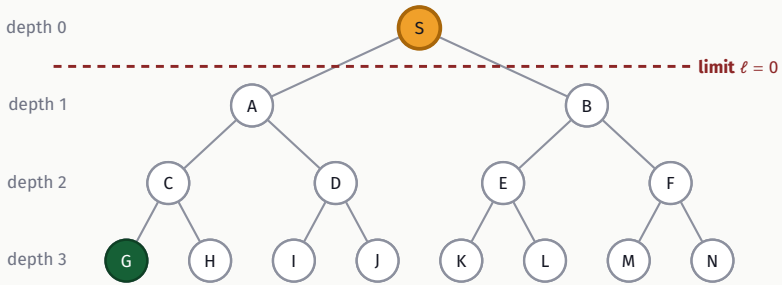
```
1 for ℓ = 0, 1, 2, ...
2     r ← DLS(root, problem, ℓ)
3     if r ≠ cutoff then return r
```

Three lines. Run depth-limited search at every limit in turn, and throw the whole search away each time.

It gets BFS's answer (the shallowest goal, hence optimal for unit costs) at DFS's memory cost, $O(bd)$. The obvious objection is that it redoes all the shallow work d times over. Hold that objection for two slides.

© 2026 Bogomil Stoyanov

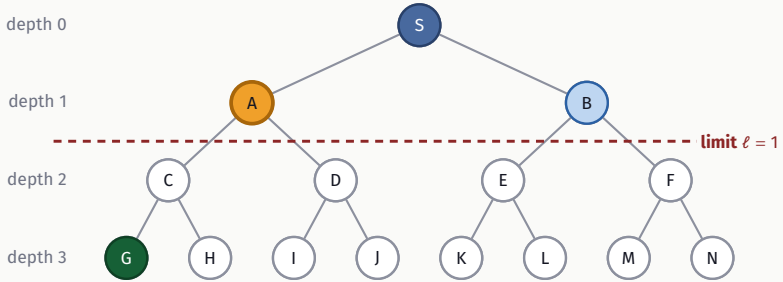
Iterative deepening, limit by limit



$\ell = 0$: test S and nothing else. Cutoff.

© 2026 Bogomil Stoyanov

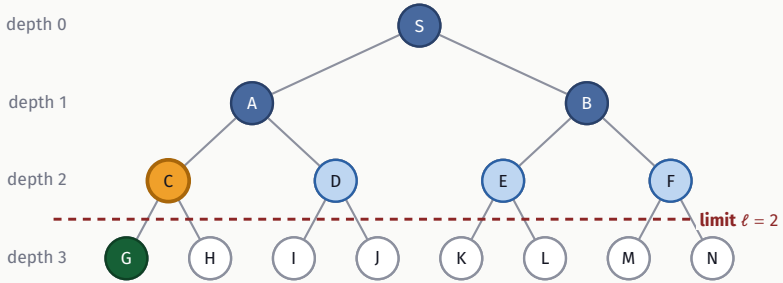
Iterative deepening, limit by limit



$\ell = 1$: start over, re-expand S, reach depth 1. Cutoff.

© 2026 Bogomil Stoyanov

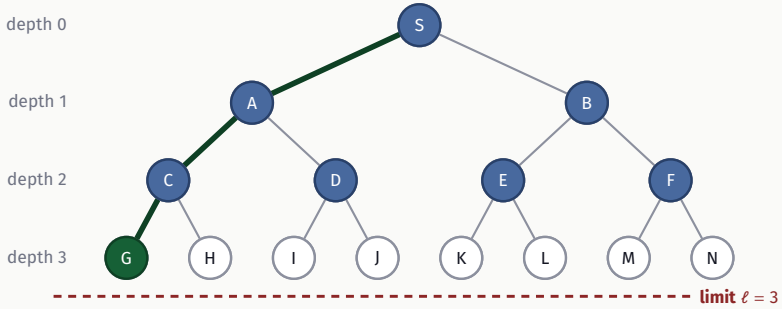
Iterative deepening, limit by limit



$\ell = 2$: start over *again*, reach depth 2. Cutoff.

© 2026 Bogomil Stoyanov

Iterative deepening, limit by limit



$\ell = 3$: the goal is found, and because we tried the limits in increasing order, it is the *shallowest* goal.

© 2026 Bogomil Stoyanov

The cost of re-searching

Nodes generated with $b = 10$, $d = 5$:

$$N(\text{IDS}) = 6 \cdot 1 + 5 \cdot 10 + 4 \cdot 100 + 3 \cdot 1000 + 2 \cdot 10^4 + 1 \cdot 10^5 = \mathbf{123\,450}$$

$$N(\text{BFS}) = 1 + 10 + 100 + 1000 + 10^4 + 10^5 + 10^6 = \mathbf{1\,111\,100}$$

© 2026 Bogomil Stoyanov

The cost of re-searching

Nodes generated with $b = 10$, $d = 5$:

$$N(\text{IDS}) = 6 \cdot 1 + 5 \cdot 10 + 4 \cdot 100 + 3 \cdot 1000 + 2 \cdot 10^4 + 1 \cdot 10^5 = \mathbf{123\,450}$$

$$N(\text{BFS}) = 1 + 10 + 100 + 1000 + 10^4 + 10^5 + 10^6 = \mathbf{1\,111\,100}$$

IDS generates *fewer* nodes than BFS here, not more. The rework is cheap because most of a tree's nodes sit in its bottom level: the deepest level alone outnumbers everything above it. Re-expanding all the shallow levels d times costs about 11% extra; BFS's habit of generating one level too deep costs far more.

© 2026 Bogomil Stoyanov

The cost of re-searching

Nodes generated with $b = 10$, $d = 5$:

$$N(\text{IDS}) = 6 \cdot 1 + 5 \cdot 10 + 4 \cdot 100 + 3 \cdot 1000 + 2 \cdot 10^4 + 1 \cdot 10^5 = \mathbf{123\,450}$$

$$N(\text{BFS}) = 1 + 10 + 100 + 1000 + 10^4 + 10^5 + 10^6 = \mathbf{1\,111\,100}$$

IDS generates *fewer* nodes than BFS here, not more. The rework is cheap because most of a tree's nodes sit in its bottom level: the deepest level alone outnumbers everything above it. Re-expanding all the shallow levels d times costs about 11% extra; BFS's habit of generating one level too deep costs far more.

In general the overhead factor is $b/(b-1)$, which *shrinks* as the branching factor grows.

© 2026 Bogomil Stoyanov

Iterative deepening: the verdict

© 2026 Bogomil Stoyanov

Iterative deepening: the verdict

Complete? Yes, if b is finite. It eventually tries every depth.

© 2026 Bogomil Stoyanov

Iterative deepening: the verdict

Complete?	Yes, if b is finite. It eventually tries every depth.
Optimal?	Yes for unit step costs, for the same reason as BFS: shallowest goal first.

© 2026 Bogomil Stoyanov

Iterative deepening: the verdict

Complete?	Yes, if b is finite. It eventually tries every depth.
Optimal?	Yes for unit step costs, for the same reason as BFS: shallowest goal first.
Time	$O(b^d)$: asymptotically the same as BFS.

© 2026 Bogomil Stoyanov

Iterative deepening: the verdict

Complete?	Yes, if b is finite. It eventually tries every depth.
Optimal?	Yes for unit step costs, for the same reason as BFS: shallowest goal first.
Time	$O(b^d)$: asymptotically the same as BFS.
Space	$O(bd)$: the same as DFS. This is the point.

© 2026 Bogomil Stoyanov

Iterative deepening: the verdict

Complete?	Yes, if b is finite. It eventually tries every depth.
Optimal?	Yes for unit step costs, for the same reason as BFS: shallowest goal first.
Time	$O(b^d)$: asymptotically the same as BFS.
Space	$O(bd)$: the same as DFS. This is the point.

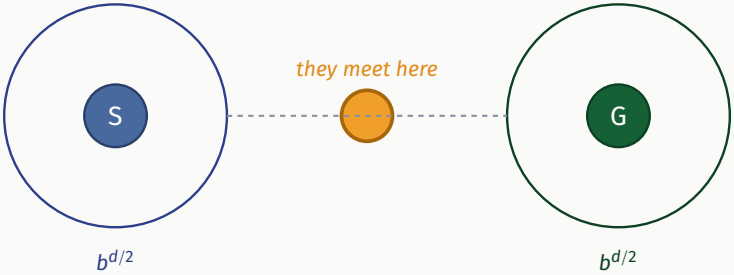
> IDS is the default uninformed search: BFS's answer at DFS's memory cost. When the step costs differ, the same trick applied to *cost* instead of depth gives iterative-deepening A^*, covered next session.

© 2026 Bogomil Stoyanov

Searching from both ends

© 2026 Bogomil Stoyanov

Bidirectional search



Two breadth-first searches at once, forward from S and backward from G, stopping when the frontiers meet.

$2b^{d/2} \ll b^d$: with $b = 10$, $d = 6$, that is 2000 nodes instead of 10^6.

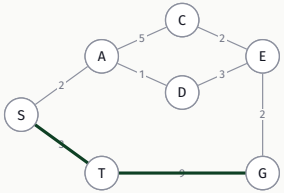
Rarely applicable: you must be able to compute *predecessors*; you need a small explicit set of goal states ("checkmate" will not do); and both frontiers stay in memory, so $O(b^{d/2})$ is still exponential.

© 2026 Bogomil Stoyanov

Summary

© 2026 Bogomil Stoyanov

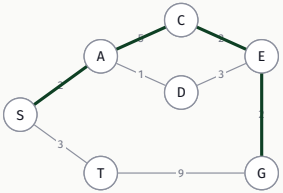
The same graph, three different answers



BFS

S, *T*, *G*, 2 edges

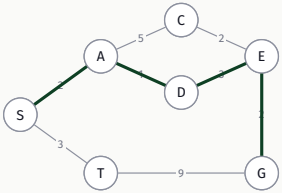
cost **12**



DFS

S, *A*, *C*, *E*, *G*, 4 edges

cost **11**



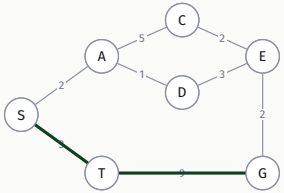
UCS

S, *A*, *D*, *E*, *G*, 4 edges

cost **8**

© 2026 Bogomil Stoyanov

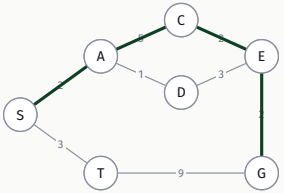
The same graph, three different answers



BFS

S, *T*, *G*, 2 edges

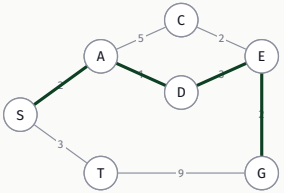
cost **12**



DFS

S, *A*, *C*, *E*, *G*, 4 edges

cost **11**



UCS

S, *A*, *D*, *E*, *G*, 4 edges

cost **8**

> Same problem, same successor order. What changed was *which node we took out of the frontier*, and that decided both the answer and whether it was any good.

© 2026 Bogomil Stoyanov

Summary of algorithms

	BFS	**UCS**	**DFS**	**DLS**	**IDS**
Complete?	Yes[a]	Yes[b]	No[c]	No[d]	Yes[a]
Optimal?	Yes[e]	**Yes**	No	No	Yes[e]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$\mathbf{O(bm)}$	$O(b\ell)$	$\mathbf{O(bd)}$
Frontier	FIFO queue	priority queue	LIFO stack	LIFO stack	LIFO stack

[a] if b is finite
[b] if every step cost $\geq \varepsilon > 0$
[c] on a finite space, with an explored set
[d] only if $\ell \geq d$
[e] only for unit step costs

© 2026 Bogomil Stoyanov

Summary of algorithms

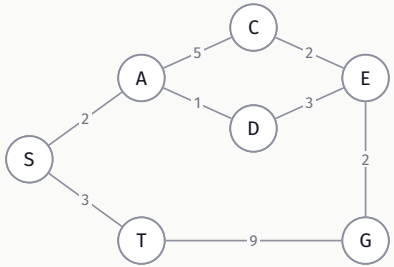
	BFS	**UCS**	**DFS**	**DLS**	**IDS**
Complete?	Yes[a]	Yes[b]	No[c]	No[d]	Yes[a]
Optimal?	Yes[e]	**Yes**	No	No	Yes[e]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$\mathbf{O(bm)}$	$O(b\ell)$	$\mathbf{O(bd)}$
Frontier	FIFO queue	priority queue	LIFO stack	LIFO stack	LIFO stack

[a] if b is finite
[b] if every step cost $\geq \varepsilon > 0$
[c] on a finite space, with an explored set
[d] only if $\ell \geq d$
[e] only for unit step costs

Choosing a search amounts to choosing a data structure for the frontier. Every row above follows from the last one.

© 2026 Bogomil Stoyanov

The missing ingredient

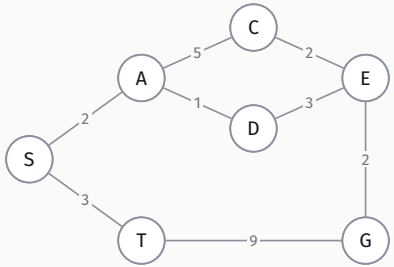


BFS and UCS both expanded *T*, the node behind the most expensive edge in the graph. None of today's algorithms had any way of knowing that *G* lies to the right.

They were handed the problem and nothing else. That is the definition of *uninformed*.

© 2026 Bogomil Stoyanov

The missing ingredient



BFS and UCS both expanded *T*, the node behind the most expensive edge in the graph. None of today's algorithms had any way of knowing that *G* lies to the right.

They were handed the problem and nothing else. That is the definition of *uninformed*.

> Next session we give the search one extra number per state, an **estimate of the remaining distance to the goal**, and it solves the same graph while touching far fewer nodes. That is **informed search**, and A^*.

© 2026 Bogomil Stoyanov